# Project 3: Musical Squares

Technical Reference Document

---

Nathaniel Lasher
EE526 - Wearable and Implantable Sensors / The University at Buffalo

Version 0.1 · October 31, 2026

# Table of Contents

1 Project Overview ..... 1
1.1 Purpose ..... 1
1.2 Scope ..... 1
1.3 Requirements ..... 1
1.4 System Summary ..... 1
1.5 Revision History ..... 2
2 Repository Layout ..... 3
2.1 `rgbTCS34725.hpp` / `rgbTCS34725.cpp` ..... 5
2.1.1 Functions ..... 5
2.1.1.1 `initSensor()` ..... 5
2.1.1.2 `writeReg()` ..... 5
2.1.1.3 `readReg()` ..... 5
2.1.1.4 `readColor()` ..... 6
2.2 main.cpp ..... 6
3 Hardware ..... 7
3.1 Components ..... 7
3.2 Pin Assignments ..... 7
3.3 Power ..... 7
4 Block and Schematic Diagrams ..... 8
4.1 Block Diagram ..... 8
4.2 Schematic ..... 9
5 References ..... 10

# List of Figures

Figure 1 System block diagram. . . . . 8
Figure 2 Circuit schematic. . . . . 9

# List of Listings

Listing 1 Repository layout. . . . . 3

# 1 Project Overview

## 1.1 Purpose

This project, realized for EE526 - Wearable and Implantable Sensors, is a project that I have had the idea for on my mind for a while. The purpose was to build your own unique smart toy device(s) using Arduino. I chose to try and work with sensors and protocols I haven't used before to add another layer to it. I was also facinated by the inner workings and split between photodiodes and LEDs.

## 1.2 Scope

The scope of this project includes the following aspects explicitly. This was more of a implementation checklist for organizing my thoughts at the start.

## 1.3 Requirements

| ID | Requirement | Status |
|---|---|---|
| R1 | RGB Color sensors will capture ambient light data | Required |
| R1.1 | System will use the I2C protocol to communicate with the TCS34725 RGB Color sensor | Planned |
| R1.2 | System must react using RGB color sensors to colored tiles in real time | Planned |
| R1.3 | System will be capable of toggling the built-in LED illumination of the color sensors on/off in software | Planned |
| R2 | System will output tones over DAC to 2 small speakers | Planned |
| R2.1 | System will map 8 tones to 8 individual colors | Planned |
| R2.2 | System will be capable of playing a note for each of the 3 sensor tracks | Required |
| R2.2.1 | System will be capable of playing multi-note chords | Planned |
| R2.3 | System will be able to 'glyssando' linearly between each of the 8 planned tones | Want |
| R3 | System will provide debug output over serial UART to a PC during normal operation | Required |
| R3.1 | Debug output will include the initialization feedback | Planned |
| R3.2 | Debug output will include the readings from each of the RGB color sensors | Planned |
| R3.3 | Debug output will include the output note / DAC information | Planned |

## 1.4 System Summary

This section is a short description of how the pieces fit together. The System block diagram is shown in Figure 1 and the schematic detailed in Section 3. The main microcontroller is an Arduino Giga R1 Wifi driving the following; 2 5W speakers via DAC and a Class-D amplifier breakout for the audio output, and 3 TCS34725 RGB Color sensors via individual I2C busses. The Power supply supplies the Giga and then all other devices are powered on the 3.3V out pins provided by it. The reason for the overkill MCU is pinout real-estate. Each of the RGB color sensors have the same exact I2C address, making a single bus impossible. DAC cannot drive the speakers alone, hence the inclusion of the amplifier with a true system reference ground for interference mitigation.

### 1.5 Revision History

| Version | Date | Changes |
|---|---|---|
| 0.1 | October 31st, 2026 | Initial draft |

## 2 Repository Layout

This section provides a brief description of repository layout, source file roles and each function included in the project, each subsection corresponds to a separate source file. Each module is a header and implementation pair; `main.cpp` has no header. Below is a tree diagram of the repo layout and a table describing source roles.

```
./
├── Code/
│   ├── musical-squares/
│   │   ├── include/
│   │   │   └── README
│   │   ├── lib/
│   │   │   └── README
│   │   ├── src/
│   │   │   ├── main.cpp
│   │   │   ├── rgbTCS34725.cpp
│   │   │   └── rgbTCS34725.hpp
│   │   ├── test/
│   │   │   └── README
│   │   ├── compile_commands.json
│   │   └── platformio.ini
│   └── Testbenches/
│       ├── giga-blink/
│       │   ├── include/
│       │   │   └── README
│       │   ├── lib/
│       │   │   └── README
│       │   ├── src/
│       │   │   └── main.cpp
│       │   ├── test/
│       │   │   └── README
│       │   ├── compile_commands.json
│       │   └── platformio.ini
│       └── rgb-sensor-debug/
│           ├── include/
│           │   └── README
│           ├── lib/
│           │   └── README
│           ├── src/
│           │   ├── main.cpp
│           │   ├── rgbTCS34725.cpp
│           │   └── rgbTCS34725.hpp
│           ├── test/
│           │   └── README
│           ├── compile_commands.json
│           └── platformio.ini
├── Documentation/
│   ├── Presentation/
│   └── Technical-reference/
│       ├── Images/
│       │   └── IMG_2811.jpeg
│       ├── tech-docs-ms.pdf
│       └── tech-docs-ms.typ
├── map-repo.sh*
└── README.md
```

Listing 1: Repository layout.

| Module | Responsibility |
|---|---|
| `rgbTCS34725.hpp` | Structure definitions, constants and function prototypes for .cpp counterpart |
| `rgbTCS34725.cpp` | Interface functions for I2C driven TCS34725 RGB color sensor |

| Module | Responsibility |
| --- | --- |
| `main.cpp` | Entry point: `setup()` and `loop()` |

## 2.1 `rgbTCS34725.hpp` / `rgbTCS34725.cpp`

This pair defines and implements the RGB color sensing.

| File | Contents |
|---|---|
| `rgbTCS34725.hpp` | Declares the `Color` data structure etc, function prototypes, and constants for RGB sensors |
| `rgbTCS34725.cpp` | Implements the 'driver' for these sensors. Including Read, Write, and Batched-Read color sampling from registers. |
| Depends on | `Wire.h`, `stdint.h` |

### 2.1.1 Functions

The following functions exist as a simple hardware API for interfacing with the I2C driven configuration, control, and data registers on the TCS34725 RGB color sensor.

#### 2.1.1.1 `initSensor()`

```
bool initSensor(TwoWire &bus);
```

Initializes the I2C bus and RGB sensor it is called upon. Call once for each connected bus-sensor pair from `setup()`. The enumeration in the paired .hpp file must be updated, but `NUM_SENSORS` can be used as the limit for a for loop iteration over this function to get the job done.

**Defined in:** `rgbTCS34725.cpp`

**Returns:** A `bool` whose value is `True` if init success; `False` otherwise. Note: expects the TCS34725 specifically and will return false for any other correctly connected sensor. See definition.

#### 2.1.1.2 `writeReg()`

```
void writeReg(TwoWire &bus, uint8_t reg, uint8_t val);
```

Writes to a given register address given an I2C bus to preform the operation on. ADDR is the same for all sensors and unessecary to pass. Commands a static register address for a single write operation (of `val` to `reg`) using the effective message (`CMD` | `reg`).

**Defined in:** `rgbTCS34725.cpp`

**Returns:** Nothing.

#### 2.1.1.3 `readReg()`

```
uint8_t readReg(TwoWire &bus, uint8_t reg);
```

Reads a given register address given an I2C bus to preform the operation on. ADDR is the same for all sensors and unessecary to pass. Commands a static register address for a single read operation using the effective message (`CMD` | `reg`) and subsequent bus read.

**Defined in:** `rgbTCS34725.cpp`

**Returns:** A `uint8_t` whose value represents the byte contents of a `reg`

#### 2.1.1.4 `readColor()`

```
Color readColor(TwoWire &bus);
```

Writes to a given register address given an I2C bus to preform the operation on. ADDR is the same for all sensors and unessecary to pass. Commands a static register address for a single write operation using the effective message (`CMD` | `reg`).

**Defined in:** `rgbTCS34725.cpp`

**Returns:** A `Color` struct containing four usable `uint16_t` halfwords for each of the combined Low and High bytes for each of the colors' registers. See rgbTCS34725.hpp

## 2.2 main.cpp

# 3 Hardware

## 3.1 Components

| Part | Description | Qty | Interface | Cost |
|---|---|---|---|---|
| Arduino Giga R1 Wifi | STM32 based microcontroller with 3 I2C lanes and DAC. | 1 | USB, JTAG, UART | $72.82 |
| TCS-34725 | RGB Color/Light Sensor | 1 | I2C | $18.02 |
| PAM8403 | Dual Channel Class-D Amplifier Break-out Board | 1 | DAC | $5.99 |
| Speaker | Generic Amazon 5W 8 Ohm speaker | 2 | Analog | $7.85 |

## 3.2 Pin Assignments

| Pin | Signal | Connected to |
|---|---|---|
| `D2` | `SDA` | TODO |
| `D3` | `SCL` | TODO |

## 3.3 Power

TODO: Supply voltage, regulators, and estimated current draw. Currently wall power.

# 4 Block and Schematic Diagrams

## 4.1 Block Diagram

Initial hardware planning block diagram.

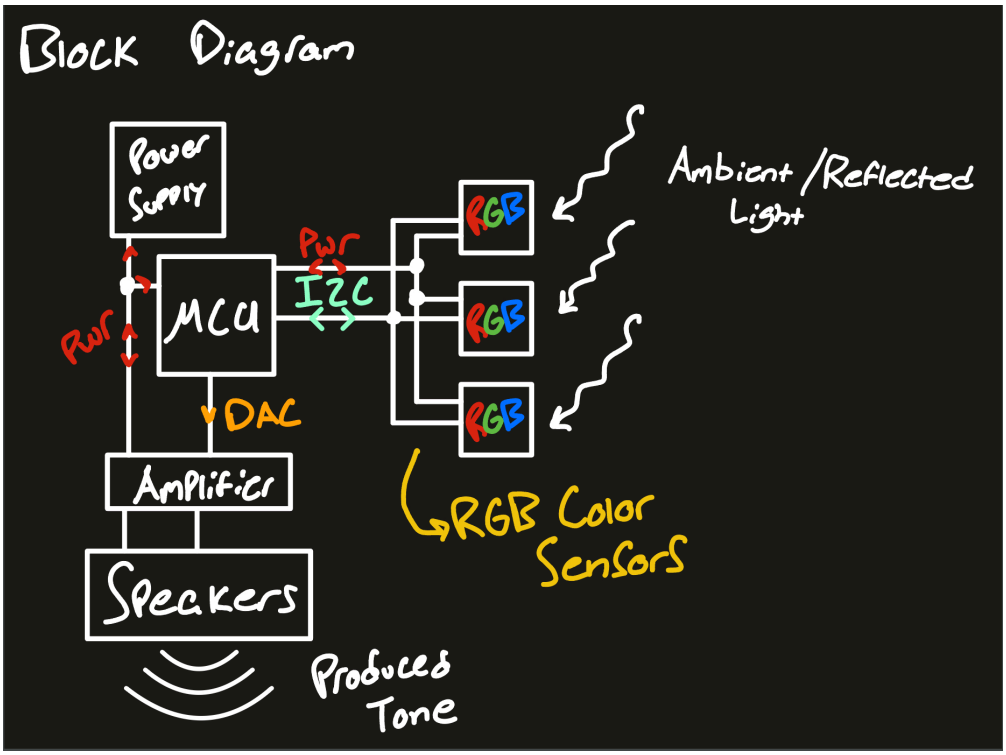



Figure 1: System block diagram.

## 4.2 Schematic

Hardware schematic showing the pin connections, circuit components and power routing.




Figure 2: Circuit schematic.

# 5 References

TODO: This section should include a full list of references. Note that I setup typst, emacs, and platformio for the first time with this project so I'm including that material.

— Typst (official) — https://typst.app/docs/ https://typst.app/docs/reference/ https://typst.app/docs/tutorial/ https://typst.app/docs/reference/syntax/ https://typst.app/docs/reference/scripting/ https://typst.app/docs/reference/styling/ https://typst.app/docs/reference/model/figure/ https://typst.app/docs/reference/model/outline/ https://typst.app/docs/reference/model/table/ https://typst.app/docs/reference/text/raw/ https://typst.app/docs/reference/layout/page/ https://typst.app/docs/reference/data-loading/read/ https://typst.app/universe/ https://github.com/typst/typst/releases

— Typst (community / editor tooling) — https://sitandr.github.io/typst-examples-book/ https://github.com/Ziqi-Yang/typst-ts-mode https://zff.dev/meow_king/typst-ts-mode/wiki/Installation https://github.com/Myriad-Dreamin/tinymist https://github.com/uben0/tree-sitter-typst

— PlatformIO — https://docs.platformio.org/ https://docs.platformio.org/en/stable/integration/ide/emacs.html https://github.com/zachmassia/platformio-mode https://bit.ly/package-manager-ioerror https://raw.githubusercontent.com/platformio/platformio-core/develop/platformio/assets/system/99-platformio-udev.rules https://community.platformio.org/t/arduino-giga-r1-wifi-board-configuration/34409 https://community.platformio.org/t/giga-dfu-issues/41982

— Emacs — https://www.gnu.org/software/emacs/manual/ https://www.gnu.org/software/emacs/manual/html_node/use-package/ https://www.gnu.org/software/emacs/manual/html_mono/eglot.html https://www.gnu.org/software/emacs/manual/html_node/elisp/index.html https://www.gnu.org/software/auctex/manual/auctex/index.html https://melpa.org/

— Language servers and language modes — https://clangd.llvm.org/ https://pkg.go.dev/golang.org/x/tools/gopls https://github.com/dominikh/go-mode.el https://github.com/python-lsp/python-lsp-server https://github.com/chipsalliance/verible https://github.com/latex-lsp/texlab

— RGB Color Sensor documentation TCS3472X datasheet (TAOS135, August 2012), https://cdn-shop.adafruit.com/datasheets/TCS34725.pdf